

CM10 - Dynamic Memory, Structs, Lists, Hash Tables

Heap allocation, user-defined types, linked lists, hash-table dictionary

Louis Ledoux

2026-07-05

Dynamic Memory Allocation

Dynamic Memory Allocation

Live pacing: about 14 min

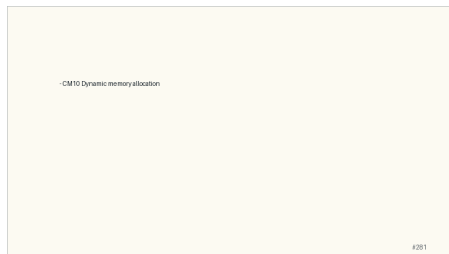
Question. When stack objects are not enough, who owns memory and structure?

Core claim. Dynamic allocation gives control over lifetime, but ownership must be made explicit in the design.

Target capability. Students can pair allocation/free, define structs, and reason about linked/hash structures.

Main Path

- ▶ Start from the question: When stack objects are not enough, who owns memory and structure?
- ▶ Build the model: Dynamic allocation gives control over lifetime, but ownership must be made explicit in the design.
- ▶ End with a checkable action: Students can pair allocation/free, define structs, and reason about linked/hash structures.
- ▶ Keep only this slide on the horizontal path during the live lecture.



Worked Example

- ▶ Use this as the live trace: Allocate one node, link it into a list, then remove it and state exactly who frees it.
- ▶ Representative source slide: 289
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.



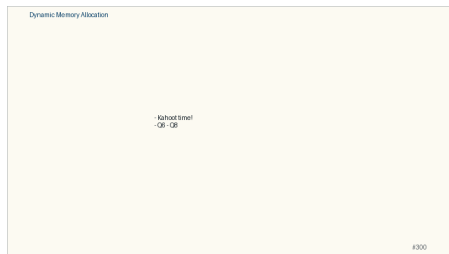
Anecdote Or Motivation

Every `malloc` creates a future obligation. Good C code makes that obligation visible.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.



Backup Index

Original slides 281-300

- CM10 Dynamic memory allocation

Backup: CM10 Dynamic memory allocation

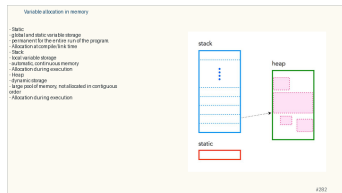
Original slide 281

- CM10 Dynamic memory allocation

Backup: Variable allocation in memory

Original slide 282

- ▶ Static:
 - ▶ global and static variable storage
 - ▶ permanent for the entire run of the program.
 - ▶ Allocation at compile/link time
- ▶ Stack:
 - ▶ local variable storage
 - ▶ automatic, continuous memory
 - ▶ Allocation during execution
- ▶ Heap:
 - ▶ dynamic storage
 - ▶ large pool of memory, not allocated in contiguous order
 - ▶ Allocation during execution



Backup: Static & Automatic memory allocation

Original slide 283

- ▶ Static:
 - ▶ allocated when the program starts
 - ▶ Variable lifetime is the entire program
- ▶ Automatic:
 - ▶ allocated during execution (function calls)
 - ▶ managed by the OS and the compiler
 - ▶ Variable lifetime is the block where they are defined
 - ▶ The variable size is fixed
 - ▶ Usually stored on the stack

Static & Automatic memory allocation

- Static:
 - allocated when the program starts
 - its scope extends to the entire program
- Automatic:
 - allocated during execution (function calls)
 - managed by the OS and the compiler
 - its scope is limited to the block where they are defined
 - The variable size is fixed
 - Usually stored on the stack

#283

Backup: Limitations & inefficiencies

Original slide 284

- ▶ Not always possible to know the exact number of array elements. May depend:
- ▶ Parameters of program execution
- ▶ Information provided by the user during execution
- ▶ Limited control over the lifetime of the variables
- ▶ Worst case: Stack overflow
- ▶ Example: Recursive functions

Limitations & inefficiencies

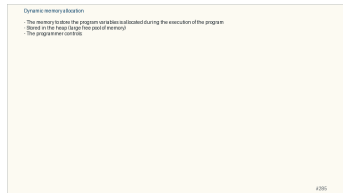
- Not always possible to know the exact number of array elements. May depend
- Parameters of program execution
- Information provided by the user during execution
- Limited control over the lifetime of the variables
- Worst case: Stack overflow
- Example: Recursive functions

#284

Backup: Dynamic memory allocation

Original slide 285

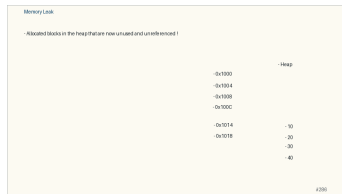
- ▶ The memory to store the program variables is allocated during the execution of the program
- ▶ Stored in the heap (large free pool of memory)
- ▶ The programmer controls:
 - ▶ the exact size (instantiated during execution)
 - ▶ Lifetime of memory locations
 - ▶ Worst-case: Memory leaks!
 - ▶ Problems: Dangling pointers!
 - ▶ ...



Backup: Memory Leak

Original slide 286

- ▶ Allocated blocks in the heap that are now unused and unreferenced !
- ▶ Heap
- ▶ 0x1000
- ▶ 0x1004
- ▶ 0x1008
- ▶ 0x100C
- ▶ 0x1014
- ▶ 10
- ▶ 0x1018
- ▶ 20
- ▶ 30
- ▶ 40



Backup: Memory Leak

Original slide 287

- ▶ Improper use of dynamic memory -> causes the memory consumption to be increased with the time
- ▶ In JAVA, garbage collector takes care of the not removed reserved parts in the heap !
- ▶ In C, the programmer is responsible for that!
- ▶ For EACH allocation -> We NEED a DEALLOCATION

Memory Leak

- improper use of dynamic memory -> causes the memory consumption to be increased with the time
- in JAVA, garbage collector takes care of the not removed reserved parts in the heap!
- in C, the programmer is responsible for that!

- For EACH allocation -> We NEED a DEALLOCATION

4287

Backup: Dangling pointer

Original slide 288

- ▶ An object is deleted or de-allocated
- ▶ If the value of the pointer is not modified
- ▶ the pointer still points to the memory location of the deleted/de-allocated memory.
- ▶ Heap/Stack
- ▶ 0x100C
- ▶ 0x1014
- ▶ 10
- ▶ 0x1018
- ▶ 20
- ▶ 30
- ▶ 40



Backup: Dynamic memory: Functions

Original slide 289

Directive:

```
#include <stdlib.h>
```

Allocation

```
malloc()
```

```
calloc()
```

```
realloc()
```

Deallocation:

```
free()
```



Backup: Dynamic allocation: malloc()

Original slide 290

Syntax:

```
void *malloc(size_t size);
```

Function

Reserves a block with a size equal to size bytes in

size_t is unsigned integer type of at least 16 bit

The content of the memory block it is NOT initialized

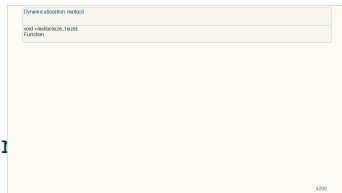
To calculate the required memory size to store an object we should use the f

The actual size of data object is compiler and target machine depended

Return value:

pointer of void type with the address of the first reserved byte (void*)

We have to explicitly cast the pointer to the type of data we want to store(



Backup: Dynamic de-allocation: free(...)

Original slide 291

Syntax:

```
void free(void *pointer);
```

Function

free() frees the memory block pointed out by the pointer

If the pointer has not been obtained by one of these functions, it is not freed

If the pointer ptr is NULL, nothing happens

No return value

The C programmer is responsible for setting free the memory, when it is no more needed



Backup: Dynamic allocation: Example

Original slide 292

Dynamic allocation: Example

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int* p;
    int i;
    int taille = 100;
    p = (int*)(malloc (taille * sizeof(int)));
    if (p == NULL)
    {
        printf("Allocation impossible:");
        exit(EXIT_FAILURE);
    }
    for (i=0; i<taille;i++){ p[i]=0; }
    free(p);
}
```

Backup: Dynamic Memory Allocation

Original slide 293

- ▶ Kahoot time!
- ▶ Q1 - Q5

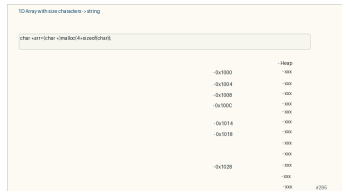


Backup: 1D Array with size characters -> string

Original slide 295

```
char *arr=(char *)malloc(4*sizeof(char));
```

- ▶ Heap
- ▶ xxx
- ▶ 0x1000
- ▶ xxx
- ▶ 0x1004
- ▶ xxx
- ▶ 0x1008
- ▶ xxx
- ▶ 0x100C
- ▶ xxx
- ▶ xxx
- ▶ 0x1014
- ▶ xxx
- ▶ 0x1018



Backup: String declaration: Remarks

Original slide 296

Dynamic declaration

```
char* s=(char*)malloc(sizeof(char) * SIZE);
```

```
s = "hello";
```

We can assign a `char` pointer to another `char` pointer

```
char *p; p=s;
```

We can assign another set of characters

```
s="bye";
```

String-declaration Remarks

Dynamic declaration
char * s=(char *)malloc(sizeof(char) * SIZE);
s = "hello";
We can assign a char pointer to another char pointer
char *p; p=s;
We can assign another set of characters
s="bye";

296

Backup: 2D Array: Single pointer

Original slide 297

```
type *arr= (type *)malloc(rows * cols * sizeof(type))
```

```
int *arr=(int *)malloc(4*3*sizeof(int));
```

► Heap

▶ XXX

▶ 0x1000

▶ XXX

► 0x1004

▶ XXX

▶ 0x1008

▶ XXX

► 0x100C

▶ XXX

▶ XXX

▶ 0x1014

```
20 Array: Single pointer
type = int * type = int[allocSize + 1] * sizeof type[1]
int *arr = int * malloc(4+3*sizeof(int))

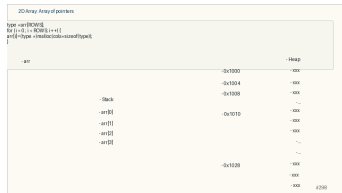
-Heap
-0x1900 -xxx
-0x1904 -xxx
-0x1908 -xxx
-0x190C -xxx
-0x1914 -xxx
-0x1918 -xxx
-xxx
-0x1928 -xxx
-xxx
-xxx
#297
```

Backup: 2D Array: Array of pointers

Original slide 298

```
type *arr[ROWS];  
for (i = 0 ; i < ROWS; i++) {  
    arr[i]=(type *)malloc(cols*sizeof(type));  
}
```

- ▶ Heap
- ▶ arr
- ▶ xxx
- ▶ 0x1000
- ▶ xxx
- ▶ 0x1004
- ▶ xxx
- ▶ 0x1008
- ▶ Stack
- ▶ ...
- ▶ xxx

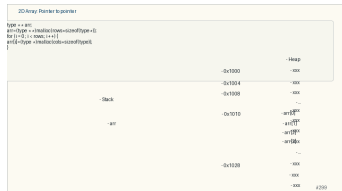


Backup: 2D Array: Pointer to pointer

Original slide 299

```
type ** arr;  
arr=(type **)malloc(rows*sizeof(type*));  
for (i = 0 ; i < rows ; i++) {  
arr[i]=(type *)malloc(cols*sizeof(type));  
}
```

- ▶ Heap
- ▶ xxx
- ▶ 0x1000
- ▶ xxx
- ▶ 0x1004
- ▶ xxx
- ▶ 0x1008
- ▶ Stack
- ▶ ...
- ▶ xxx



Backup: Dynamic Memory Allocation

Original slide 300

- ▶ Kahoot time!
- ▶ Q6 - Q8



User-defined Data Types

User-defined Data Types

Live pacing: about 8 min

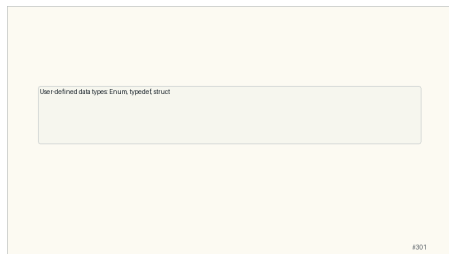
Question. When stack objects are not enough, who owns memory and structure?

Core claim. Dynamic allocation gives control over lifetime, but ownership must be made explicit in the design.

Target capability. Students can pair allocation/free, define structs, and reason about linked/hash structures.

Main Path

- ▶ Start from the question: When stack objects are not enough, who owns memory and structure?
- ▶ Build the model: Dynamic allocation gives control over lifetime, but ownership must be made explicit in the design.
- ▶ End with a checkable action: Students can pair allocation/free, define structs, and reason about linked/hash structures.
- ▶ Keep only this slide on the horizontal path during the live lecture.



Worked Example

- ▶ Use this as the live trace: Allocate one node, link it into a list, then remove it and state exactly who frees it.
- ▶ Representative source slide: 302
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

Enumerated type declaration & definition

- Syntax:
enum identifier {element1,...}
- Function:
The enumeration creates a data type that groups a set of elements that behave as integral constants
- Declares a new enumeration type with the name enum identifier
- A variable is defined as enum identifier, it can be assigned one of the elements
- Example:
enum day {mon, tue, wed, thu, fri, sat, sun};
mon is coded by 0, tue by 1, ..., sun par 6
- This is a new type called enum day
- enum day today;
today=mon;

#302

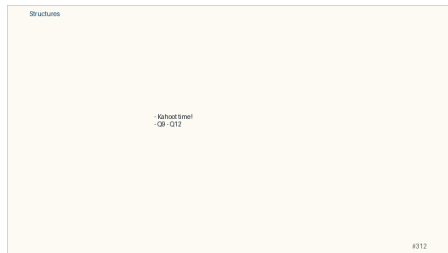
Anecdote Or Motivation

Every `malloc` creates a future obligation. Good C code makes that obligation visible.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.



Backup Index

Original slides 301-312

User-defined data types: Enum, typedef, struct

Backup: User-defined data types: Enum, typedef, struct

Original slide 301

User-defined data types: Enum, typedef, struct

Backup: Enumerated type declaration & definition

Original slide 302

- ▶ Syntax:
- ▶ `enum identifier {element1,...}`
- ▶ Function:
- ▶ The enumeration creates a data type that groups a set of elements that behave as integral constants
- ▶ Declares a new enumeration type with the name `enum identifier`
- ▶ A variable is defined as `enum identifier`, it can be assigned one of the elements
- ▶ Example:
- ▶ `enum day{mon, tue, wen, thu, fri, sat, sun} ;`
- ▶ `mon` is coded by 0, `tue` by 1, ..., `sun` par 6
- ▶ This is a new type called `enum day`
- ▶ `enum day today;`
- ▶ `today=wen;`

Enumerated type declaration & definition

- Syntax:
`enum identifier {element1,...}`
- Function:
- The enumeration creates a data type that groups a set of elements that behave as integral constants
- Declares a new enumeration type with the name `enum identifier`
- A variable is defined as `enum identifier`, it can be assigned one of the elements
- Example:
`enum day{mon, tue, wen, thu, fri, sat, sun};`
`mon` is coded by 0, `tue` by 1, ..., `sun` par 6
This is a new type called `enum day`
`enum day today;`
`today=wen;`

4302

Backup: Structure type declaration

Original slide 303

Syntax

```
struct identifier {  
member definition;  
  
...  
};
```

Function:

A structure creates a data type that groups items of possibly different type

Declares a new structure type named **struct identifier**

The members of the structure can be integral, **float**, arrays, pointers.

Example:

```
struct address{  
char street[100];  
int number;  
};
```

► NO space is reserved!

Structure type declaration

Syntax
struct identifier {
member definition;

};

Function
A structure creates a data type that groups items of possibly different types into a single type.
Declares a new structure type named struct identifier.
The members of the structure can be integral, float, arrays, pointers.
Example:
struct address {
char street[100];
int number;
};

from by creating a new type that describes the members of the group.

4303

Backup: Structure type declaration

Original slide 304

- ▶ The structure type declaration CANNOT be initialized!
- ▶ This is a new data type declaration, not a structure variable definition!
- ▶ Only when the structure variable is defined !
- ▶ The corresponding memory is reserved
- ▶ Now we can initialize the members of the structure

Structure type declaration

The structure type declaration CANNOT be initialized!
This is a new data type declaration, not a structure variable definition!
Only when the structure variable is defined !
The corresponding memory is reserved
Now we can initialize the members of the structure

304

Backup: Structure variable definition & initialisation

Original slide 305

Separately from the structure declaration:

```
struct address MyAddress = {"Paul Bert", 12};
```

During structure declaration:

```
struct address{  
    char street[100];  
    int number;  
} MyAddress = {"Paul Bert", 12};
```

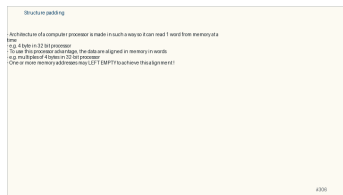
- ▶ Space is reserved!
- ▶ Equal to the one required to store the structure members aligned

Structure variable definition & initialisation
Separately from the structure declaration: struct address MyAddress = {"Paul Bert", 12}; During structure declaration: struct address { char street[100]; int number; } MyAddress = {"Paul Bert", 12}; Space is reserved! Equal to the one required to store the structure members aligned
405

Backup: Structure padding

Original slide 306

- ▶ Architecture of a computer processor is made in such a way so it can read 1 word from memory at a time
- ▶ e.g. 4 byte in 32 bit processor
- ▶ To use this processor advantage, the data are aligned in memory in words
- ▶ e.g. multiples of 4 bytes in 32-bit processor
- ▶ One or more memory addresses may LEFT EMPTY to achieve this alignment !

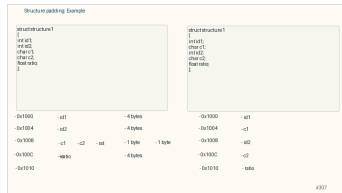


Backup: Structure padding: Example

Original slide 307

```
struct structure1
{
    int id1;
    int id2;
    char c1;
    char c2;
    float ratio;
};
```

```
struct structure1
{
    int id1;
    char c1;
    int id2;
    char c2;
    float ratio;
};
```



Backup: Structure variable: Access members

Original slide 308

```
struct address{  
char street[100];  
int number;  
};
```

Through a structure variable: .

```
struct address MyAddress;  
strcpy(MyAddress.street, "Paul Bert");  
MyAddress.number=12;
```

Through a pointer to the structure variable: ->

```
struct address MyAddress;  
struct address *MyAddressPtr = &MyAddress;  
strcpy(MyAddressPtr->street, "Paul Bert");  
MyAddressPtr->number=12;
```

Structure variable: Access members

```
struct address  
{char street[100];  
int number;  
};
```

Through a structure variable:
struct address MyAddress;
strcpy(MyAddress.street, "Paul Bert");
MyAddress.number=12;
Through a pointer to the structure variable ->
struct address *MyAddressPtr = &MyAddress;
strcpy(MyAddressPtr->street, "Paul Bert");
MyAddressPtr->number=12;

4308

Backup: Definition of a new type

Original slide 309

Syntax

```
typedef type IDENTIFIER;
```

Examples

```
typedef int LENGHT;
```

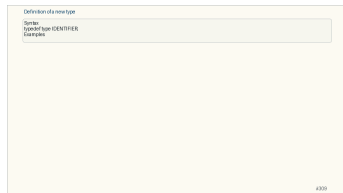
```
typedef enum {false, true} BOOLEAN ;
```

```
typedef struct date date;
```

Use of typedef to avoid repetition of keyword **struct**

Remark

There is no identifier for the **enum** in the definition of the type **BOOLEAN**



Backup: Structure type: Examples

Original slide 310

Structure type: Examples

```
struct date /* déclaration du type struct date */
{
    short j ;
    short m ;
    int a ;
} ;
```

```
/* déclaration du type struct UNE_DATE */
typedef enum {lun, mar, mer, jeu, ven, sam, dim} JOUR ;
typedef struct
{
    JOUR j ;
    struct date d ;
} UNE_DATE ;
```

Backup: Access struct members

Original slide 311

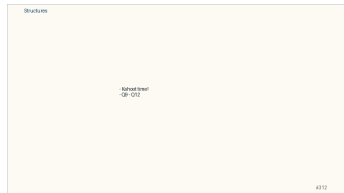
- ▶ Variable structure `UNE_DATE foo = { mer, {3, 9, 2014} } ; foo.j = mer ; foo.d.a = 2014 ;`
- ▶ Structure pointer `UNE_DATE* queljour = &foo; queljour->j = foo.j;`
- ▶ `queljour->d.a = 2014;`



Backup: Structures

Original slide 312

- ▶ Kahoot time!
- ▶ Q9 - Q12



Lists

Lists

Live pacing: about 16 min

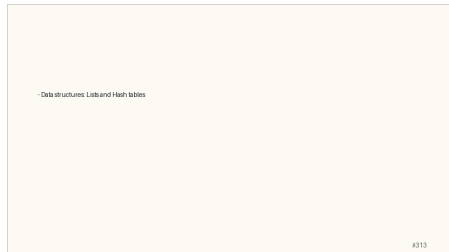
Question. When stack objects are not enough, who owns memory and structure?

Core claim. Dynamic allocation gives control over lifetime, but ownership must be made explicit in the design.

Target capability. Students can pair allocation/free, define structs, and reason about linked/hash structures.

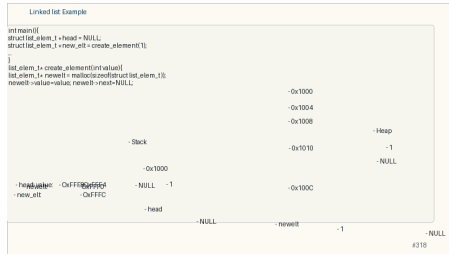
Main Path

- ▶ Start from the question: When stack objects are not enough, who owns memory and structure?
- ▶ Build the model: Dynamic allocation gives control over lifetime, but ownership must be made explicit in the design.
- ▶ End with a checkable action: Students can pair allocation/free, define structs, and reason about linked/hash structures.
- ▶ Keep only this slide on the horizontal path during the live lecture.



Worked Example

- ▶ Use this as the live trace: Allocate one node, link it into a list, then remove it and state exactly who frees it.
- ▶ Representative source slide: 318
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.



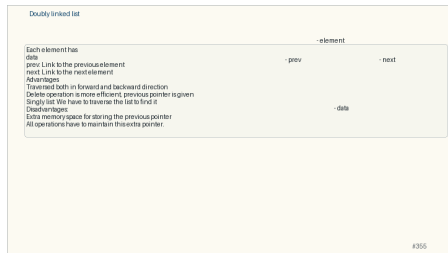
Anecdote Or Motivation

Every `malloc` creates a future obligation. Good C code makes that obligation visible.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.



Backup Index

Original slides 313-355

- Data structures: Lists and Hash tables

Backup: Data structures: Lists and Hash tables

Original slide 313

- Data structures: Lists and Hash tables

Backup: Linked lists Vs arrays

Original slide 314

- ▶ Linear data structure (like arrays)
- ▶ Linked list elements NOT stored in contiguous location
- ▶ are linked using pointers
- ▶ element
- ▶ element
- ▶ element
- ▶ element
- ▶ Head
- ▶ Tail
- ▶ NULL



Backup: Why Linked list?

Original slide 315

- ▶ Array limitations:
- ▶ Fixed size: We must know the upper limit on the number of elements in advance
- ▶ Insert/Delete an element: Expensive
- ▶ Create room by shifting existing elements !
- ▶ Ex. : Add new element 101 in an array with sorted ids
- ▶ `id[]=[100, 103, 105, 200]`
- ▶ List Advantages:
- ▶ Dynamic size
- ▶ Ease of insertion/deletion
- ▶ List Drawbacks:
- ▶ Random access is not allowed
- ▶ Access elements sequentially
- ▶ Extra memory space required for the pointer to each element

Why Linked list?

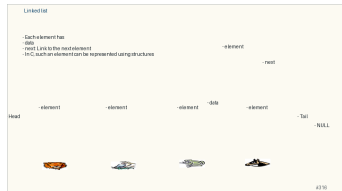
- Array limitations
- Fixed size: We must know the upper limit on the number of elements in advance
- Insert/Delete an element: Expensive
- Create room by shifting existing elements !
- Ex. Add new element 101 in an array with sorted ids
- `id[]=[100, 103, 105, 200]`
- List Advantages
- Dynamic size
- Ease of insertion/deletion
- List Drawbacks
- Random access is not allowed
- Access elements sequentially

43/55

Backup: Linked list

Original slide 316

- ▶ Each element has
- ▶ data
- ▶ next: Link to the next element
- ▶ In C, such an element can be represented using structures
- ▶ element
- ▶ next
- ▶ data
- ▶ element
- ▶ element
- ▶ element
- ▶ element
- ▶ Head
- ▶ Tail
- ▶ NULL



Backup: Linked list: Example

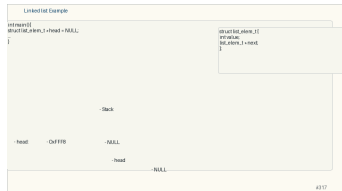
Original slide 317

```
int main(){
    struct list_elem_t *head = NULL;

    ...
}
```

```
struct list_elem_t {
    int value;
    list_elem_t *next;
};
```

- ▶ Stack
- ▶ head:
- ▶ 0xFFF8
- ▶ NULL
- ▶ head
- ▶ NULL



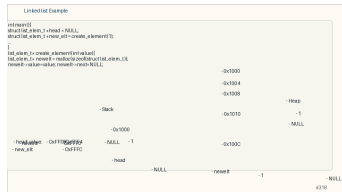
Backup: Linked list: Example

Original slide 318

```
int main(){
    struct list_elem_t *head = NULL;
    struct list_elem_t *new_elt = create_element(1);
    ...
}

list_elem_t* create_element(int value){
    list_elem_t* newelt = malloc(sizeof(struct list_elem_t ));
    newelt->value=value; newelt->next=NULL;
}
```

- ▶ 0x1000
- ▶ 0x1004
- ▶ 0x1008
- ▶ Heap
- ▶ Stack
- ▶ 1
- ▶ 0x1010



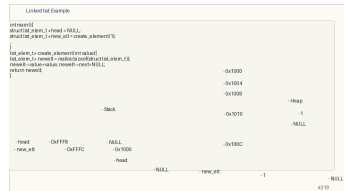
Backup: Linked list: Example

Original slide 319

```
int main(){
    struct list_elem_t *head = NULL;
    struct list_elem_t *new_elt = create_element(1);
    ...
}

list_elem_t* create_element(int value){
    list_elem_t* newelt = malloc(sizeof(struct list_elem_t ));
    newelt->value=value; newelt->next=NULL;
    return newelt;
}
```

- ▶ 0x1000
- ▶ 0x1004
- ▶ 0x1008
- ▶ Heap
- ▶ Stack

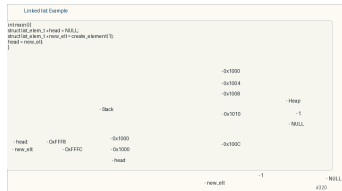


Backup: Linked list: Example

Original slide 320

```
int main(){
    struct list_elem_t *head = NULL;
    struct list_elem_t *new_elt = create_element(1);
    head = new_elt;
}
```

- ▶ 0x1000
- ▶ 0x1004
- ▶ 0x1008
- ▶ Heap
- ▶ Stack
- ▶ 1
- ▶ 0x1010
- ▶ NULL
- ▶ 0x1000
- ▶ head:



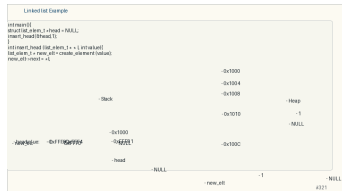
Backup: Linked list: Example

Original slide 321

```
int main(){
    struct list_elem_t *head = NULL;
    insert_head(&head, 1);
}

int insert_head (list_elem_t * * l, int value){
    list_elem_t * new_elt = create_element (value);
    new_elt->next = *l;
```

- ▶ 0x1000
- ▶ 0x1004
- ▶ 0x1008
- ▶ Stack
- ▶ Heap
- ▶ 1
- ▶ 0x1010
- ▶ NULL



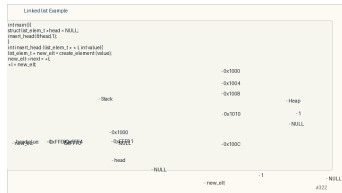
Backup: Linked list: Example

Original slide 322

```
int main(){
    struct list_elem_t *head = NULL;
    insert_head(&head, 1);
}

int insert_head (list_elem_t * * l, int value){
    list_elem_t * new_elt = create_element (value);
    new_elt->next = *l;
    *l = new_elt;
}
```

- ▶ 0x1000
- ▶ 0x1004
- ▶ 0x1008
- ▶ Stack
- ▶ Heap
- ▶ 1
- ▶ 0x1010



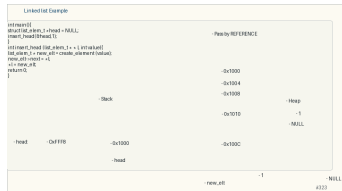
Backup: Linked list: Example

Original slide 323

```
int main(){
    struct list_elem_t *head = NULL;
    insert_head(&head, 1);
}

int insert_head (list_elem_t * * l, int value){
    list_elem_t * new_elt = create_element (value);
    new_elt->next = *l;
    *l = new_elt;
    return 0;
}
```

- ▶ Pass by REFERENCE
- ▶ 0x1000
- ▶ 0x1004
- ▶ 0x1008
- ▶ Stack



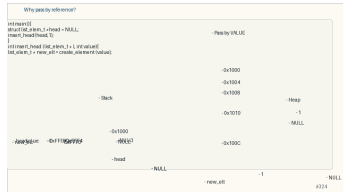
Backup: Why pass by reference?

Original slide 324

```
int main(){
    struct list_elem_t *head = NULL;
    insert_head(head, 1);
}

int insert_head (list_elem_t * l, int value){
    list_elem_t * new_elt = create_element (value);
```

- ▶ Pass by VALUE
- ▶ 0x1000
- ▶ 0x1004
- ▶ 0x1008
- ▶ Stack
- ▶ Heap
- ▶ 1
- ▶ 0x1010
- ▶ NULL



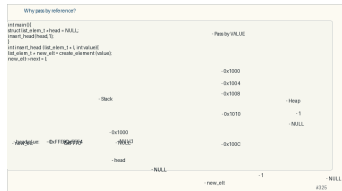
Backup: Why pass by reference?

Original slide 325

```
int main(){
    struct list_elem_t *head = NULL;
    insert_head(head, 1);
}

int insert_head (list_elem_t * l, int value){
    list_elem_t * new_elt = create_element (value);
    new_elt->next = l;
```

- ▶ Pass by VALUE
- ▶ 0x1000
- ▶ 0x1004
- ▶ 0x1008
- ▶ Stack
- ▶ Heap
- ▶ 1
- ▶ 0x1010



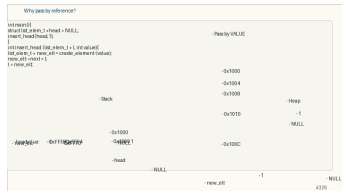
Backup: Why pass by reference?

Original slide 326

```
int main(){
    struct list_elem_t *head = NULL;
    insert_head(head, 1);
}

int insert_head (list_elem_t * l, int value){
    list_elem_t * new_elt = create_element (value);
    new_elt->next = l;
    l = new_elt;
}
```

- ▶ Pass by VALUE
- ▶ 0x1000
- ▶ 0x1004
- ▶ 0x1008
- ▶ Stack
- ▶ Heap
- ▶ 1



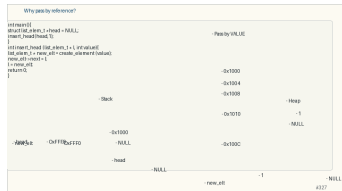
Backup: Why pass by reference?

Original slide 327

```
int main(){
    struct list_elem_t *head = NULL;
    insert_head(head, 1);
}

int insert_head (list_elem_t * l, int value){
    list_elem_t * new_elt = create_element (value);
    new_elt->next = l;
    l = new_elt;
    return 0;
}
```

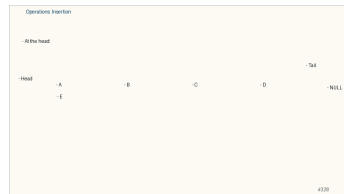
- ▶ Pass by VALUE
- ▶ 0x1000
- ▶ 0x1004
- ▶ 0x1008
- ▶ Stack



Backup: Operations: Insertion

Original slide 328

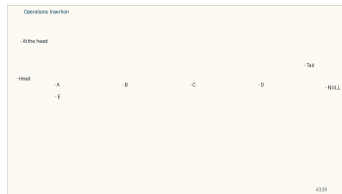
- ▶ At the head:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ E



Backup: Operations: Insertion

Original slide 329

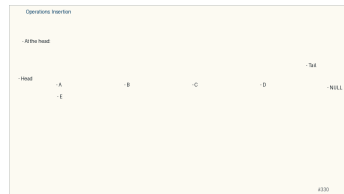
- ▶ At the head:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ E



Backup: Operations: Insertion

Original slide 330

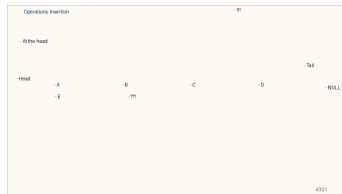
- ▶ At the head:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ E



Backup: Operations: Insertion

Original slide 331

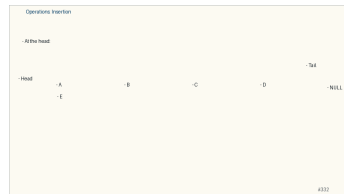
- ▶ !!!
- ▶ At the head:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ E
- ▶ ???



Backup: Operations: Insertion

Original slide 332

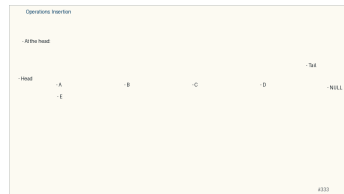
- ▶ At the head:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ E



Backup: Operations: Insertion

Original slide 333

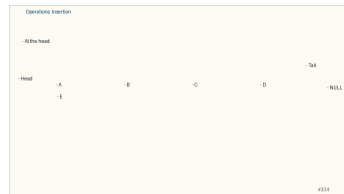
- ▶ At the head:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ E



Backup: Operations: Insertion

Original slide 334

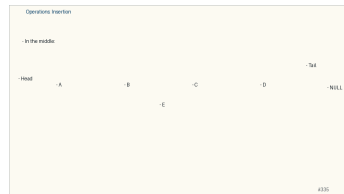
- ▶ At the head:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ E



Backup: Operations: Insertion

Original slide 335

- ▶ In the middle:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ E



Backup: Operations: Insertion

Original slide 336

- ▶ In the middle:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ E

Backup: Operations: Insertion

Original slide 338

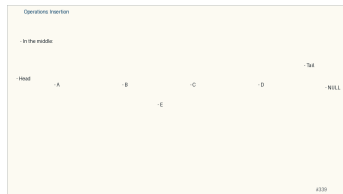
- ▶ In the middle:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ ???
- ▶ E



Backup: Operations: Insertion

Original slide 339

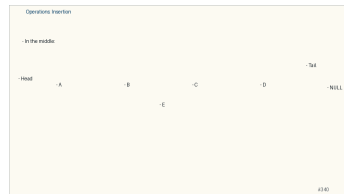
- ▶ In the middle:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ E



Backup: Operations: Insertion

Original slide 340

- ▶ In the middle:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ E



Backup: Operations: Insertion

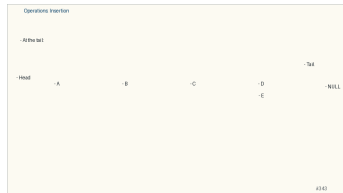
Original slide 342

- ▶ In the middle:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ E

Backup: Operations: Insertion

Original slide 343

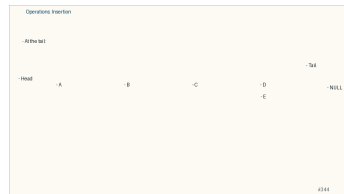
- ▶ At the tail:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ E



Backup: Operations: Insertion

Original slide 344

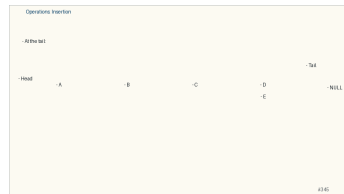
- ▶ At the tail:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ E



Backup: Operations: Insertion

Original slide 345

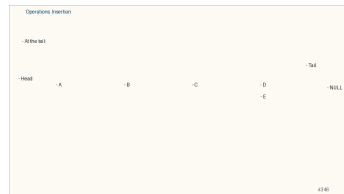
- ▶ At the tail:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ E



Backup: Operations: Insertion

Original slide 346

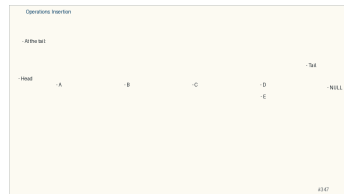
- ▶ At the tail:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ E



Backup: Operations: Insertion

Original slide 347

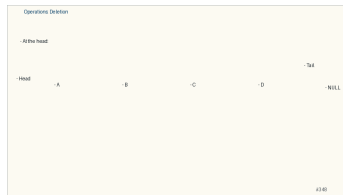
- ▶ At the tail:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ E



Backup: Operations: Deletion

Original slide 348

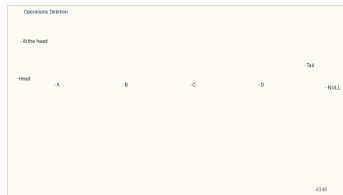
- ▶ At the head:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL



Backup: Operations: Deletion

Original slide 349

- ▶ At the head:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL



Backup: Operations: Deletion

Original slide 350

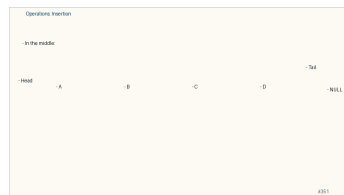
- ▶ At the head:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ ATTENTION: Free the memory space used for element A
- ▶ Need to know the pointer to the element A!



Backup: Operations: Insertion

Original slide 351

- ▶ In the middle:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL



Backup: Operations: Insertion

Original slide 352

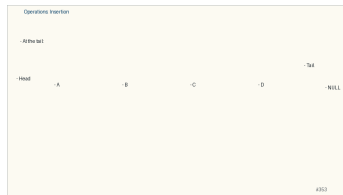
- ▶ In the middle:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ ATTENTION: Free the memory space used for element B
- ▶ Need to know the pointer to the element B!



Backup: Operations: Insertion

Original slide 353

- ▶ At the tail:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL



Backup: Operations: Insertion

Original slide 354

- ▶ At the tail:
- ▶ Tail
- ▶ Head
- ▶ A
- ▶ B
- ▶ C
- ▶ D
- ▶ NULL
- ▶ ATTENTION: Free the memory space used for element D
- ▶ Need to know the pointer to the element C!



Backup: Doubly linked list

Original slide 355

► element

Each element has
data

prev: Link to the previous element

next: Link to the next element

Advantages

Traversed both in forward and backward direction

Delete operation is more efficient, previous pointer is given

Singly list: We have to traverse the list to find it

Disadvantages:

Extra memory space for storing the previous pointer

All operations have to maintain this extra pointer.

► prev

► next

► data



Hash Tables

Hash Tables

Live pacing: about 11 min

Question. When stack objects are not enough, who owns memory and structure?

Core claim. Dynamic allocation gives control over lifetime, but ownership must be made explicit in the design.

Target capability. Students can pair allocation/free, define structs, and reason about linked/hash structures.

Main Path

- ▶ Start from the question: When stack objects are not enough, who owns memory and structure?
- ▶ Build the model: Dynamic allocation gives control over lifetime, but ownership must be made explicit in the design.
- ▶ End with a checkable action: Students can pair allocation/free, define structs, and reason about linked/hash structures.
- ▶ Keep only this slide on the horizontal path during the live lecture.

- Hash Table

- An array of a fixed size *hsize*
- An element is associated with an integer value called hash value
- a number within the range 0 to *hsize* - 1
- depicts in which cell of the hash table the element will be stored (key)
- calculated using a Hash Function
- Maps a data set of arbitrary size to a data set of fixed size
- unique value
- Fast access of the data
- Search in the index indicated by the key

#356

Worked Example

- ▶ Use this as the live trace: Allocate one node, link it into a list, then remove it and state exactly who frees it.
- ▶ Representative source slide: 366
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

- Hash table Dictionary

```
hash_table = hash_ptr;
hash_ptr = (hash_table *) malloc(sizeof(hash_table));
```

Hash table allocation:
malloc reserves a memory space equal to the structure hash_table
hash_ptr will point to a structure containing an integer and a pointer to a word_list

- Hash table size initialisation
- We have finished the hash table declaration?
hash_ptr->size = PRIME_NUMBER

#366

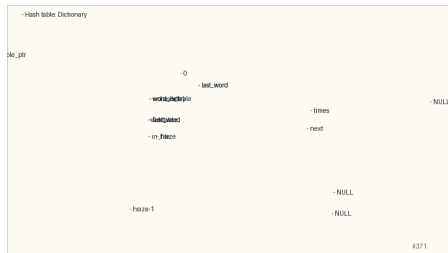
Anecdote Or Motivation

Every `malloc` creates a future obligation. Good C code makes that obligation visible.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.



Backup Index

Original slides 356-371

- Hash Table

- An array of a fixed size $hsize$
- An element is associated with an integer value called hash value
- a number within the range 0 to $hsize - 1$
- depicts in which cell of the hash table the element will be stored (key)
- calculated using a Hash Function
- Maps a data set of arbitrary size to a data set of fixed size
- unique value
- Fast access of the data:
- Search in the index indicated by the key

Backup: Hash Table

Original slide 356

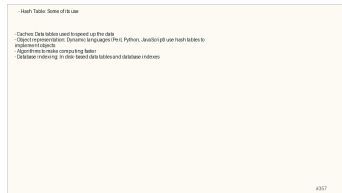
- ▶ An array of a fixed size $hsize$
- ▶ An element is associated with an integer value called hash value
- ▶ a number within the range 0 to $hsize - 1$
- ▶ depicts in which cell of the hash table the element will be stored (key)
- ▶ calculated using a Hash Function
- ▶ Maps a data set of arbitrary size to a data set of fixed size
- ▶ unique value
- ▶ Fast access of the data:
- ▶ Search in the index indicated by the key



Backup: Hash Table: Some of its use

Original slide 357

- ▶ Caches: Data tables used to speed up the data
- ▶ Object representation: Dynamic languages (Perl, Python, JavaScript) use hash tables to implement objects
- ▶ Algorithms to make computing faster
- ▶ Database indexing: In disk-based data tables and database indexes



Backup: Hash Function: Simple examples

Original slide 358

Integers:

Return key MOD hsize

Strings:

We add up the ASCII values of the chars, and return

```
int hash( char *key, int hsize )
{
    int hash_val = 0;
    while( *key != '\0' )
        hash_val += *key++;
    return( hash_val % hsize );
}
```



Backup: Collisions

Original slide 359

- ▶ When more than one element have the same hash value !
- ▶ Example:
- ▶ $hsize = 10$
- ▶ Elements: the first 10 perfect squares
- ▶ 0, 1, 4, 9, 16, 25, 36, 49, 64, 81
- ▶ hash function: $hash(x) = x \bmod 10$

-Collisions

-When more than one element have the same hash value !

-Example:

- hsize = 10
- Elements: the first 10 perfect squares
- 0, 1, 4, 9, 16, 25, 36, 49, 64, 81
- hash function: hash(x) = x mod 10

4359

Backup: Collisions: Solution

Original slide 360

- ▶ Separate chaining (open hashing):
- ▶ Each element of the hash table is a linked list
- ▶ Collision: Store both elements in the same linked list
- ▶ 0
- ▶ NULL
- ▶ 1
- ▶ 81
- ▶ NULL
- ▶ 4
- ▶ 64
- ▶ NULL
- ▶ 25
- ▶ NULL
- ▶ 16
- ▶ 36
- ▶ NULL

Collision Solution			
- Separate chaining (open hashing): - Each element of the hash table is a linked list - Collision: Store both elements in the same linked list			
-3	-NULL		
-1	-81	-NULL	
-4	-64	-NULL	
-25	-NULL		
-16	-36	-NULL	
-9	-49	-NULL	#390

Backup: Problems

Original slide 361

The distribution of the values may NOT be even !

Example

hsize = 10007

Variable: strings of 8 ASCII characters

The hash function: adds up the ASCII values of the

-> it can only assume values 0 - 1016, which is 127*8

- Problems

The distribution of the values may NOT be even !

Example
hsize = 10007
Variable: strings of 8 ASCII characters
The hash function adds up the ASCII values of the characters and returns the modulus operation
-> it can only assume values 0 - 1016, which is 127*8

4301

Backup: Hash table: Dictionary

Original slide 362

- ▶ Create a dictionary and search engine:
- ▶ Read files word by word
- ▶ Add to the dictionary one word found in a file and the number of times found in the file
- ▶ Search for a word in the dictionary
- ▶ Use hash table and linked lists to store the words with the same key

- Hash table: Dictionary

- Create a dictionary and search engine:
- Read file word by word
- Add to the dictionary one word found in a file and the number of times found in the file
- Search for a word in the dictionary
- Use hash table and linked lists to store the words with the same key

#362

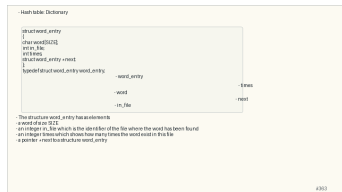
Backup: Hash table: Dictionary

Original slide 363

```
struct word_entry
{
    char word[SIZE];
    int in_file;
    int times;
    struct word_entry *next;
};

typedef struct word_entry word_entry;
```

- ▶ word_entry
- ▶ times
- ▶ word
- ▶ next
- ▶ in_file
- ▶ The structure word_entry has as elements
- ▶ a word of size SIZE



Backup: Hash table: Dictionary

Original slide 364

```
struct word_list
{
    struct word_entry *first_word;
    struct word_entry *last_word;
};
typedef struct word_list word_list;
```

- ▶ last_word
- ▶ word_list
- ▶ first_word
- ▶ The structure word_list has as elements
- ▶ a pointer *first_word to a structure word_entry
- ▶ a pointer *last_word to a structure word_entry



Backup: Hash table: Dictionary

Original slide 365

```
struct hash_table
{
word_list *htable;
int hsize;
};
typedef struct hash_table hash_table;
```

- ▶ hash_table
- ▶ htable
- ▶ The hash_table structure contains
- ▶ a pointer htable to a type word_list
- ▶ A pointer to a pointer of the structure word_entry
- ▶ the fixed size hsize
- ▶ We have finished the hash table declaration?
- ▶ hsize



Backup: Hash table: Dictionary

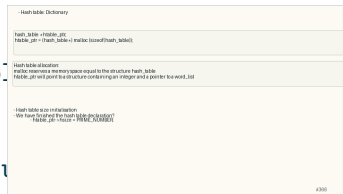
Original slide 366

```
hash_table *htable_ptr;  
htable_ptr = (hash_table*) malloc (sizeof(hash_table));
```

Hash table allocation:

malloc reserves a memory space equal to the structure
htable_ptr will point to a structure containing an integer and a pointer to

- ▶ Hash table size initialisation
- ▶ We have finished the hash table declaration?
- ▶ `htable_ptr->hsize = PRIME_NUMBER;`



Backup: Hash table: Dictionary

Original slide 367

- ▶ `htable_ptr`
- ▶ `hash_table`
- ▶ `htable`
- ▶ `hsize`



Backup: Hash table: Dictionary

Original slide 368

- ▶ Hash table pointer initialisation:
- ▶ We need to allocate space for the linked lists.
- ▶ We use an array of structures word_list equal to the table size

```
htable_ptr->htable =  
(word_list*) malloc(sizeof(word_list) * htable_ptr->hsize );
```

- ▶ We need to initialize the pointers of the structures word_list to NULL

```
for(i = 0; i < htable_ptr->hsize ; i++ ){  
    htable_ptr->htable[i].first_word = NULL;  
    htable_ptr->htable[i].last_word = NULL;  
}
```

Hash table: Dictionary

- Hash table pointer initialisation:
- We need to allocate space for the linked lists.
- We use an array of structures word_list equal to the table size

```
htable_ptr->htable =  
(word_list*) malloc(sizeof(word_list) * htable_ptr->hsize );
```

- We need to initialize the pointers of the structures word_list to NULL

```
for(i = 0; i < htable_ptr->hsize ; i++ ){  
    htable_ptr->htable[i].first_word = NULL;  
    htable_ptr->htable[i].last_word = NULL;  
}
```

4398

Backup: Hash table: Dictionary

Original slide 369

- ▶ htable_ptr
- ▶ NULL
- ▶ 0
- ▶ NULL
- ▶ last_word
- ▶ last_word
- ▶ word_list
- ▶ word_list
- ▶ hash_table
- ▶ first_word
- ▶ first_word
- ▶ htable
- ▶ hsize
- ▶ NULL
- ▶ hsize-1
- ▶ NULL



Backup: Hash table: Dictionary

Original slide 370

- ▶ When a new word is added:
- ▶ We need to allocate space for the word
- ▶ Connected to the list based on the hash key

```
htable_ptr->htable[key].first_word =  
(word_entry*)malloc(sizeof(word_entry));
```

-Hash table: Dictionary

- When a new word is added:
 - We need to allocate space for the word
 - Connect to the list based on the hash key

```
htable_ptr->htable[key].first_word =  
(word_entry*)malloc(sizeof(word_entry));
```

4370

Backup: Hash table: Dictionary

Original slide 371

- ▶ htable_ptr
- ▶ 0
- ▶ last_word
- ▶ last_word
- ▶ word_list
- ▶ word_entry
- ▶ word_list
- ▶ hash_table
- ▶ NULL
- ▶ times
- ▶ word
- ▶ first_word
- ▶ first_word
- ▶ htable
- ▶ next
- ▶ in_file

